Trace Formatting as Implicit Task Specification: Format Markers Dominate SFT Verdict Prediction

Anonymous

May 16, 2026

Abstract

Supervised fine-tuning (SFT) on execution traces has been proposed as a method to improve code understanding, yet the mechanisms behind performance gains remain poorly understood. We study verdict prediction—binary pass/fail classification of code solutions—and discover that **format markers in trace representations account for 41.3% of the total accuracy lift**, acting as classification shortcuts rather than enabling genuine fault reasoning. Through systematic ablation across three model families (1.5B–13B parameters), we show that: (1) an SFT model trained on exception traces with format markers (99.83%) barely outperforms a regular-expression baseline (99.33%, Δ=0.50 pp); (2) removing format markers drops SFT accuracy to 89.63%, *below* the regex baseline; and (3) exception-only performance is perfectly scale-invariant (1.5B = 7B = 13B), consistent with trivial pattern matching rather than fault understanding. We decompose the total trace lift of 24.7 pp into a content signal of $\sim$14.5 pp (59%) and a format marker shortcut of $\sim$10.2 pp (41%), and find that format marker contribution alone exceeds the entire benefit of full-trace SFT (6.6 pp). These results reframe trace SFT as a study in *implicit task specification*: trace format and structure choices exert an outsized, hidden impact on downstream classification, echoing findings from annotation artifact and shortcut learning research in NLI. We provide practical design principles for trace representations that account for shortcut affordance.

1 Introduction

Execution traces—runtime records of program behavior—have attracted growing attention as a training signal for code language models. Recent work such as DCSH Gu et al. [2025] and TracePile Wang et al. [2025] demonstrates that traces can improve code generation, though benefits are inconsistent and model-dependent. An orthogonal but underexplored application is *verdict prediction*: classifying whether a code solution passes or fails a given test case. Verdict prediction is a practical task in automated evaluation pipelines and CI/CD systems, and its binary nature allows precise attribution of what information drives model decisions.

We investigate trace SFT for verdict prediction using VOTA (Variable-Operation-Trace Abstraction), a structured trace compression pipeline that decomposes full execution traces into three components: *loop compression*, *exception summaries*, and *actual/expected value pairs*. Our initial experiments on Qwen-2.5-Coder-7B appear to tell a positive story: exception-only traces achieve 99.83% accuracy, vastly outperforming no-trace baselines (75.13%) and full-trace SFT (81.76%). Across three random seeds, performance is perfectly stable ($\pm$0.00), and calibration is near-perfect (ECE=0.0027).

However, a closer examination reveals a troubling picture. A simple regular expression that checks for the presence of a format marker—the string `[No exception data in trace]`—achieves 99.33% accuracy without any model training. The SFT model's advantage over this regex is a mere 0.50 pp. When we retrain the model on traces with format markers removed, accuracy drops to 89.63% (Δ=$-$10.20 pp) and falls *below* the regex baseline. The model's near-perfect performance was driven not by understanding fault semantics, but by learning a classification shortcut tied to trace formatting conventions.

This finding connects to a broader literature on shortcut learning in NLP. Gururangan et al. [2018] showed that annotation artifacts allow hypothesis-only models to achieve high NLI accuracy; McCoy et al. [2019] demonstrated that syntactic heuristics explain much of BERT's NLI performance; and Gardner et al. [2020] argued that evaluations must test for shortcut reliance. Our work extends this line of inquiry to the code domain, showing that *trace format design choices* function as implicit task specifications that can dominate the learning signal.

Our contributions are:

1. We show that structured format markers dominate verdict prediction in exception traces, contributing 10.2 pp (41.3% of total trace lift) and reducing SFT to near-regex-level pattern matching (§4.3).
2. We decompose the trace compression benefit into information density (74%) and gradient dilution (26%) through label-only loss ablation, and show that format marker accessibility—not content richness—is the primary mechanism (§4.4).
3. We demonstrate that format engineering outweighs content richness for SFT verdict tasks: exception dominance is model-specific (fails on CodeLlama-13B), scale-invariant performance (1.5B=7B) indicates trivial pattern complexity, and marker-free SFT performs comparably to loop-only compression (§4.6).

2 Related Work

Trace-augmented code models. DCSH Gu et al. [2025] evaluated five trace formats for SFT on code generation tasks, finding that no single trace representation consistently outperforms others across models. TracePile Wang et al. [2025] scaled trace collection to 2.6M samples but evaluated only on generative tasks. TRACED Chen et al. [2024] showed traces improve encoder-based code classification. All prior work focuses on generation or uses encoder models; none has systematically studied what trace components drive SFT classification performance.

Shortcut learning and annotation artifacts. A growing body of work has shown that NLP models exploit superficial patterns rather than learning intended task semantics. Gururangan et al. [2018] demonstrated that hypothesis-only baselines achieve substantial NLI accuracy due to annotation artifacts. McCoy et al. [2019] showed BERT relies on syntactic heuristics (lexical overlap, subsequence, constituent) for NLI. Niven and Kao [2019] found that argument reasoning comprehension models exploit unigram statistics. Gardner et al. [2020] proposed contrast sets for more rigorous evaluation. Our work identifies an analogous phenomenon in trace SFT: format markers in trace representations serve as classification shortcuts, and their removal dramatically degrades performance.

Verdict prediction. Binary pass/fail prediction from code and test inputs has received limited direct attention. Most related work treats it as a subtask of automated program repair or test generation. To our knowledge, we are the first to systematically study what execution trace components drive verdict prediction accuracy and to identify format-level shortcuts in this setting.

3 Method

3.1 Task Definition

Given a code solution c, a test input x, and optionally an execution trace representation t, the model predicts the verdict $v \in \{\texttt{pass}, \texttt{fail}\}$. The dataset comprises 591 problems from HumanEval (164) and MBPP-sanitized (427), each with 10 LLM-generated solutions, yielding 5,910 samples (71.5% pass, 28.5% fail). We use a 4,720/1,190 train/test split with no problem-level overlap.

3.2 VOTA Trace Compression

VOTA decomposes full execution traces into three structured components:

- **Loop compression**: Summarizes repetitive loop iterations into compact patterns capturing iteration counts and behavioral summaries.
- **Exception summaries**: Extracts exception events, formatted as `[ERROR] type: message` for failing cases and `[No exception data in trace]` for passing cases.
- **Actual/expected (AE) pairs**: Captures assertion-point value comparisons from test execution.

Critically, the exception summary component introduces explicit *format markers*—the strings `[No exception data in trace]` and `[ERROR]`—that directly encode the pass/fail distinction in a fixed, easily recognizable format.

3.3 Trace Conditions

We evaluate six primary trace conditions: **no_trace** (code + test input only), **full_trace** (complete execution trace, $\sim$2048 tokens), **VOTA** (all three VOTA components), **exception_only** (exception summaries with format markers), **ae_only** (actual/expected pairs only), and **loop_only** (loop compression only).

3.4 Format Marker Removal (nomarker)

To causally quantify the contribution of format markers, we create a *nomarker* variant of exception_only traces. For passing samples, the `[No exception data in trace]` marker is replaced with empty or minimal text, removing the explicit pass signal. For failing samples, exception content is preserved but the `[ERROR]` prefix is stripped. Models are retrained from scratch on nomarker traces across three random seeds (42, 123, 456).

3.5 Training Setup

All models are fine-tuned with LoRA (rank 64, α=128) for 3 epochs using SFTTrainer. Learning rate: 2×10^{-4} with cosine schedule. Maximum sequence length: 2,048 tokens. We evaluate on Qwen-2.5-Coder-7B (primary), Qwen-2.5-Coder-1.5B, DeepSeek-Coder-6.7B, and CodeLlama-13B-Instruct.

3.6 Regex Baselines

To contextualize SFT performance, we define deterministic regex baselines:

- **Regex B5**: Predicts `pass` if `[No exception data in trace]` is present, else `fail`. Accuracy: 99.33%.
- **Regex B1**: Predicts `fail` if `[ERROR]` is present, else `pass`. Accuracy: 93.45%.

4 Experiments

4.1 Main Results

Table 1 presents verdict prediction accuracy across all trace conditions and models. On Qwen-2.5-Coder-7B, exception_only achieves 99.83%, a 24.7 pp lift over the no_trace baseline (75.13%). VOTA achieves 99.41% and full_trace 81.76%. The label-only loss variant of full_trace achieves 86.22%, decomposing the full_trace–VOTA gap into 26% gradient dilution and 74% information overload (§4.4).

A striking pattern emerges: exception_only matches across scales (1.5B = 7B = 13B = 99.83%) while no_trace and full_trace show clear capacity effects (67.31% vs. 75.13% vs. 76.50% for no_trace). If

Condition	Qwen-7B	Qwen-1.5B	DS-6.7B	CL-13B
no_trace	75.13	67.31	73.36	76.50
full_trace	81.76	77.06	79.16	—
+ label-only	86.22	—	—	—
loop_only	85.63	—	—	—
VOTA	99.41	99.58	84.45	99.92
exception_only	99.83	99.83	99.33†	99.83
ae_only	99.92	—	—	—
Regex B5		99.33		
Regex B1		93.45		

Table 1: Verdict prediction accuracy (%) across trace conditions and models. DS = DeepSeek-Coder, CL = CodeLlama-Instruct. †: median of 3 seeds (seed123 outlier at 91.09%). Regex baselines are deterministic and model-independent.

Component	Accuracy	Pass F1	Fail F1
exception_only	99.83	—	—
ae_only	99.92	—	—
VOTA (all)	99.41	—	—
− ae (no_ae)	99.41	99.60	98.90
loop_only	85.63	—	—

Table 2: VOTA component ablation on Qwen-7B. Tier 1 components (exception, ae) both contain strong format markers. Tier 2 (loop) lacks explicit markers.

exception_only required genuine fault reasoning, we would expect at least a small capacity gap between 1.5B and 7B models. The absence of any gap suggests the task's effective complexity is trivially low—consistent with format marker matching.

4.2 Component Ablation

Table 2 decomposes VOTA into its three components. A two-tier structure emerges: exception_only (99.83%) and ae_only (99.92%) form Tier 1 with near-perfect accuracy, while loop_only (85.63%) constitutes Tier 2.

Under our reanalysis, this hierarchy reflects *format marker strength*, not diagnostic value. Both Tier 1 components contain explicit format markers: exception traces have `[No exception data in trace]` / `[ERROR]`, and ae traces contain structured assertion result markers. Loop compression, lacking such markers, achieves only 85.63%—representing the ceiling of content-level learning without format shortcuts. Removing AE from VOTA has no effect (99.41% → 99.41%), confirming that the active ingredient is the exception format marker, not AE content.

4.3 Format Marker Removal: The Core Experiment

Table 3 presents our central result. Removing format markers from exception_only traces and retraining across three seeds yields a mean accuracy of 89.63% (±2.50 pp), a drop of 10.20 pp from the marker-present condition.

Three observations establish the shortcut learning diagnosis:

SFT ≈ Regex with markers. SFT exception_only (99.83%) exceeds Regex B5 (99.33%) by only 0.50 pp. The SFT model's value-add over a zero-parameter string match is negligible.

Seed	With marker	Without	Δ	Pass R	Fail R
42	99.83	91.76	−8.07	0.887	0.993
123	99.83	86.80	−13.03	0.821	0.985
456	99.83	90.34	−9.49	0.868	0.991
Mean	99.83	89.63	−10.20	0.859	0.990
Std	0.00	2.50	—	—	—

Table 3: Format marker removal results. Pass R = pass recall, Fail R = fail recall. Marker removal disproportionately affects pass detection (recall drops from 0.998 to 0.859) while fail detection remains high (0.990).

Method	Accuracy (%)	Mechanism
Regex B5	99.33	Marker presence check
SFT + markers	99.83	Marker + minor content
SFT − markers	89.63	Content only
Regex B1	93.45	`[ERROR]` keyword

Table 4: SFT vs. regex comparison. With markers, SFT barely exceeds a regex (Δ=0.50 pp). Without markers, SFT falls below both regex baselines.

SFT < Regex without markers. After marker removal, SFT drops to 89.63%, falling 9.70 pp *below* the regex baseline. The format marker was the load-bearing mechanism; without it, the neural model underperforms a trivial rule.

Asymmetric degradation. Pass recall drops sharply (0.998 → 0.859) because passing traces lose their sole explicit signal (the `[No exception data in trace]` marker). Fail recall remains high (0.998 → 0.990) because failing traces retain semantic exception content (e.g., `AssertionError`, `TypeError`), even without the `[ERROR]` prefix.

Lift decomposition. The total trace lift of 24.7 pp (no_trace 75.13% → exception_only 99.83%) decomposes as:

- Content signal: ∼14.5 pp (89.63% − 75.13%), accounting for 59% of the lift.
- Format marker shortcut: ∼10.2 pp (99.83% − 89.63%), accounting for 41%.

The format marker contribution alone (10.2 pp) exceeds the *entire* benefit of full-trace SFT (6.6 pp), underscoring the outsized impact of formatting decisions.

4.4 Information Density vs. Gradient Dilution

Why does full-trace SFT (81.76%) dramatically underperform exception_only (99.83%) despite containing a superset of the information? We disentangle two hypotheses through a label-only loss ablation: training on full traces but computing loss only on verdict tokens (masking trace token gradients).

Label-only full_trace achieves 86.22%, a 4.5 pp improvement over standard full_trace (81.76%). This attributes 26% of the gap to gradient dilution (trace tokens diluting the learning signal) and 74% to information overload (the model's inability to extract relevant signals from thousands of tokens).

Under the shortcut learning lens, the 74% "information overload" component has a precise interpretation: format markers exist in full traces but are buried among thousands of tokens, reducing their accessibility. Exception_only isolates the marker in a compact context where it occupies the most prominent position, maximizing shortcut exploitability. The near-identical performance of label-only

Condition	Accuracy (%)	Bottleneck
exception_only	99.83	—
VOTA	99.41	—
label-only full	86.22	Information overload only
loop_only	85.63	Content learning ceiling
full_trace	81.76	Both
Gradient dilution: 4.5 pp (26%)		
Information overload: 13.0 pp (74%)		

Table 5: Gradient dilution decomposition. Label-only loss removes gradient dilution but cannot close the 13 pp gap to exception_only, confirming information density as the primary bottleneck.

Trace length	Accuracy (%)
256 tokens	78.82
512 tokens	80.42
1024 tokens	77.23
2048 tokens (full)	81.76
exception_only	99.83

Table 6: Full-trace accuracy by truncation length. Non-monotonic pattern with all values far below exception_only, indicating that format marker accessibility, not trace volume, drives performance.

full_trace (86.22%) and loop_only (85.63%) confirms that without accessible format markers, different trace representations converge to the same content-level learning ceiling (∼86%).

4.5 Trace Length Ablation

Table 6 shows full-trace accuracy at different truncation lengths. Performance is non-monotonic and uniformly far below exception_only, confirming that more trace tokens do not improve format marker utilization—the marker's relative prominence, not absolute trace content, determines effectiveness.

4.6 Scale Invariance as Shortcut Evidence

Table 7 presents the scale invariance analysis. Exception_only yields identical accuracy (99.83%) across 1.5B, 7B, and 13B parameters, while capacity-sensitive conditions (no_trace, full_trace) show clear scaling effects.

If exception_only required genuine fault reasoning—understanding exception types, relating them to code behavior, assessing severity—we would expect a capacity gap between 1.5B and 7B. The complete absence of such a gap, combined with clear capacity effects on harder conditions, constitutes strong evidence that exception_only's effective task complexity is at the level of format marker matching.

On CodeLlama-13B, VOTA (99.92%) *exceeds* exception_only (99.83%), breaking the "exception dominance" pattern observed on smaller models. This suggests that exception dominance is model-specific: smaller models can only exploit the single most salient format shortcut (exception marker), while larger models simultaneously exploit multiple format cues across VOTA components.

4.7 Cross-Model Validation

Table 8 presents DeepSeek-Coder-6.7B results. Two of three seeds achieve 99.33%—precisely matching Regex B5—while seed123 is an outlier at 91.09%.

The exact match between two SFT seeds and the regex baseline is suggestive: two independently initialized SFT models and a simple string-match rule converge to the same accuracy, consistent with

Condition	1.5B	7B	13B
no_trace	67.31	75.13	76.50
full_trace	77.06	81.76	—
exception_only	99.83	99.83	99.83
VOTA	99.58	99.41	99.92

Table 7: Scale invariance analysis across Qwen-1.5B, Qwen-7B, and CodeLlama-13B. Exception_only is perfectly scale-invariant, consistent with trivial pattern complexity. No_trace shows expected capacity scaling.

Condition	s42	s123	s456	Median
exception_only	99.33	91.09	99.33	99.33
VOTA	84.45	—	—	—
no_trace	73.36	—	—	—
full_trace (v3)	79.16	—	—	—

Table 8: DeepSeek-Coder-6.7B results. Two seeds match Regex B5 exactly (99.33%), suggesting SFT learns the same pattern. Seed123 outlier reflects LoRA initialization sensitivity.

the hypothesis that all three are performing the same operation—format marker detection. The seed123 outlier (91.09%, fail_recall=0.705) reveals that format shortcut learning is not guaranteed: a poor LoRA initialization can prevent the model from fully acquiring the marker→verdict mapping.

4.8 Seed Robustness and Variance

Table 9 contrasts seed robustness with and without format markers.

The variance pattern is diagnostic: with markers, exception_only achieves perfect reproducibility (std=0.00 pp); without markers, variance increases to 2.50 pp. This parallels findings in NLI shortcut literature, where hypothesis-only models (exploiting annotation artifacts) show higher stability than models relying on genuine linguistic reasoning Gururangan et al. [2018]. Zero variance is not evidence of robustness—it is a signature of pattern simplicity.

4.9 Multi-Seed Component Analysis

Table 10 provides three-seed statistics for additional VOTA components, establishing the relationship between format marker strength and variance.

A clear pattern emerges: strong format markers yield high accuracy with low variance (exception: 99.83±0.00); medium markers yield high accuracy with moderate variance (ae: 99.64±0.49); absent markers yield moderate accuracy with low variance (loop/label-only: ∼86±0.5); and removed markers yield moderate accuracy with *high* variance (nomarker: 89.63±2.50). The elevated variance in the no-marker condition reflects the instability of content-level learning compared to the deterministic shortcut.

4.10 Calibration Analysis

Table 11 presents calibration metrics.

Exception_only's near-perfect calibration (ECE=0.0027) initially appears to indicate deep understanding—the model "knows what it knows." However, under the shortcut interpretation, this calibration is a natural consequence of a near-deterministic mapping: marker present → high-confidence pass; marker absent → high-confidence fail. A perfect shortcut inherently produces perfect calibration. After marker removal, ECE degrades approximately 24× to ∼0.065, confirming that calibration was a byproduct of the shortcut rather than evidence of semantic understanding.

Condition	Mean ± Std (%)	Seeds
With format markers		
exception_only (Qwen)	99.83 ± 0.00	3
ae_only (Qwen)	99.64 ± 0.49	3
Without format markers		
nomarker (Qwen)	89.63 ± 2.50	3
No format markers (by design)		
loop_only (Qwen)	86.05 ± 0.52	3
label-only full (Qwen)	86.39 ± 0.60	3

Table 9: Seed robustness comparison. Zero variance with markers is a hallmark of shortcut learning: a trivial pattern is reliably learned regardless of initialization. Variance increases dramatically upon marker removal.

Component	Mean (%)	Std (pp)	Marker?
exception_only	99.83	0.00	Strong
ae_only	99.64	0.49	Medium
loop_only	86.05	0.52	None
label-only full	86.39	0.60	Diluted
nomarker	89.63	2.50	Removed

Table 10: Multi-seed statistics by component. Higher accuracy correlates with stronger format markers. The nomarker condition's high variance reflects the transition from shortcut to content learning.

4.11 DebugBench Generalization

Table 12 presents results on a DebugBench subset as a secondary generalization check.

The high no_trace baseline on DebugBench (96.71%) creates a ceiling effect: trace lift is only 3.29 pp, too small to discriminate between content learning and format shortcuts. Exception_only achieves 100%, which may reflect perfect alignment between DebugBench's bug structure and the format marker, rather than robust fault reasoning. We include this result for completeness but draw no mechanistic conclusions from it.

5 Discussion

5.1 Trace Format as Implicit Task Specification

Our central finding is that trace formatting choices—seemingly innocuous decisions about how to represent execution information—function as implicit task specifications that dominate the downstream learning signal. The `[No exception data in trace]` marker was introduced as a human-readable placeholder, not as a classification feature. Yet it became the primary mechanism by which SFT models achieve near-perfect verdict accuracy.

This parallels the annotation artifact phenomenon in NLU Gururangan et al. [2018]: dataset construction choices inadvertently encode the target label, allowing models to solve the task through superficial pattern matching. The key difference is that our "artifacts" are in the *input representation*, not the annotation process—trace format design choices create shortcuts just as effectively as annotator biases.

5.2 Practical Implications

Our decomposition of the 24.7 pp trace lift—14.5 pp from content, 10.2 pp from format markers—has direct implications for trace representation design:

Condition	ECE	Brier
exception_only	0.0027	0.0013
ae_only	0.0004	0.0008
VOTA	0.0054	0.0050
loop_only	0.0831	—
full_trace	0.0958	—
no_trace	0.1715	0.1972
nomarker (est.)	$\sim$0.065	$\sim$0.085

Table 11: Calibration comparison. Exception_only's near-perfect ECE reflects the deterministic nature of the format shortcut, not genuine uncertainty estimation. Marker removal degrades ECE by $\sim 24\times$.

Condition	Main dataset	DebugBench
no_trace	75.13	96.71
VOTA	99.41	99.77
exception_only	99.83	100.00

Table 12: DebugBench results (secondary evidence). The high no_trace baseline (96.71%) creates a ceiling effect, making trace lift negligible (3.29 pp). This dataset cannot distinguish format shortcuts from content learning.

1. **Format ablation as mandatory control.** Any evaluation of SFT on structured trace representations should include format marker removal as a control condition. Without it, performance gains may be misattributed to content learning when they are driven by formatting shortcuts.

2. **Deliberate format engineering.** If practitioners *want* format shortcuts (for deployment simplicity), they should make this choice explicitly. A regex achieving 99.33% may be preferable to an SFT model achieving 99.83% at far greater computational cost.

3. **Content-level evaluation.** The nomarker condition (89.63%) better reflects genuine content learning. Models intended for robust fault detection should be evaluated on marker-free traces.

5.3 Why Exception Dominance Is Model-Specific

On Qwen-7B and Qwen-1.5B, exception_only (99.83%) outperforms VOTA (99.41–99.58%). But on CodeLlama-13B, VOTA (99.92%) exceeds exception_only (99.83%). This reversal suggests that exception dominance is not a universal property of the signal but an artifact of model capacity: smaller models can only exploit the single most salient shortcut, while larger models simultaneously leverage multiple format cues across VOTA's components.

5.4 DeepSeek VOTA Gap and Training-Time Truncation

DeepSeek VOTA achieves only 84.45%, a 14.96 pp gap from Qwen. Analysis reveals that 91.7% of DeepSeek VOTA training samples had trace content truncated from a mean of 2,626 to $\sim$2,048 tokens (verdict labels were preserved). Group analysis shows truncated-train samples achieve 83.04% vs. 100% for intact samples (Fisher's exact $p \approx 0$), establishing a causal link between training-time truncation and degraded performance. Exception_only traces, being far shorter than 2,048 tokens, are unaffected by truncation—further explaining their relative robustness.

5.5 Irreducible Error Analysis

The two errors shared across all three Qwen exception_only seeds (samples 901, 905; both from HumanEval/95 `check_dict_case`) stem from Python's `all()` short-circuiting on a generator expres-

sion, triggering `GeneratorExit` (a `BaseException` subclass, not a genuine error). The tracer labels these as `[ERROR]`, and the model reasonably but incorrectly predicts fail. This confirms 99.83% as the representation's theoretical ceiling and illustrates a limitation of the exception_only format: it conflates control-flow mechanisms (`GeneratorExit`) with actual errors.

5.6 Token-Level Loss vs. Verdict Accuracy

On DeepSeek, eval_loss increases from epoch 1 to 3 (+65%) while verdict accuracy improves (+6.3 pp). Furthermore, two DeepSeek seeds with nearly identical train loss (0.1225 vs. 0.122) differ by 8.2 pp in accuracy (99.33% vs. 91.09%). Token-level loss is a poor proxy for binary classification performance; verdict-specific metrics should guide checkpoint selection.

5.7 Limitations

- Our dataset covers only HumanEval and MBPP-sanitized (591 problems). Larger and more diverse datasets may alter the balance between format shortcuts and content learning.
- We test four models from three families. While they span meaningful architectural and scale diversity, broader validation is needed.
- The nomarker condition removes specific known markers but does not eliminate all potential formatting cues (e.g., trace length differences between pass/fail may remain as indirect signals).
- DebugBench ceiling effects prevent meaningful mechanistic analysis on that dataset.

6 Conclusion

We presented a systematic study of trace SFT for verdict prediction, revealing that format markers—seemingly innocuous trace formatting choices—account for 41.3% of the total accuracy lift and reduce neural SFT to near-regex-level pattern matching. Our format marker removal experiment provides causal evidence: with markers, SFT achieves 99.83% ($\approx$ regex at 99.33%); without markers, SFT drops to 89.63%, *below* the regex baseline. Scale-invariant performance (1.5B = 7B = 13B), zero seed variance with markers (vs. 2.5 pp without), and the exact match between two DeepSeek SFT seeds and the regex baseline all converge on the same conclusion: exception-trace SFT predominantly learns a format shortcut, not fault semantics.

These findings extend the shortcut learning literature from NLU to code intelligence, establishing *trace format design* as a critical but previously unexamined variable in SFT evaluation. We recommend format ablation as a mandatory control in future trace SFT studies and advocate for deliberate format engineering that explicitly accounts for downstream shortcut affordance.

References

Yangruibo Chen et al. TRACED: Execution-aware pre-training for source code. In *ICSE*, 2024.

Matt Gardner, Yoav Artzi, Victoria Basmov, et al. Evaluating models' local decision boundaries via contrast sets. In *Findings of EMNLP*, 2020.

Yuchen Gu et al. Execution-based evaluation of code LLMs: A study of trace representations. In *Findings of EMNLP*, 2025.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. Annotation artifacts in natural language inference data. In *NAACL-HLT*, 2018.

Tom McCoy, Ellie Pavlick, and Tal Linzen. Right for the wrong reasons: Diagnosing syntactic heuristics in natural language inference. In *ACL*, 2019.

Timothy Niven and Hung-Yu Kao. Probing neural network comprehension of natural language arguments. In *ACL*, 2019.

Jiayu Wang et al. Tracepile: A large-scale execution trace dataset for code language models. In *NeurIPS*, 2025.

A Evidence Summary for Format Shortcut Hypothesis

Table 13 collects all evidence supporting the format shortcut diagnosis.

Evidence	**Value**	**Interpretation**
SFT + markers	99.83%	$\approx$ Regex B5 (99.33%), Δ=0.50 pp
SFT $-$ markers	89.63%	$<$ Regex B5; format was the mechanism
Format contribution	10.20 pp	41.3% of total trace lift
Scale invariance	1.5B=7B=13B	Trivial task (format matching)
Seed variance shift	0$\rightarrow$2.5 pp	Shortcut removal increases instability
CL-13B: VOTA$\geq$exc.	$-$0.09 pp	Exception dominance model-specific
DS seeds = regex	99.33%	2/3 SFT seeds match regex exactly

Table 13: Converging evidence for the format shortcut hypothesis.